

Date:

Exp No:8

Aim:Implement memory management schemes(first Fit, Best Fit and Worst Fit)

Theory

1.First Fit Algorithm

Theory

Definition

The **First Fit** algorithm allocates the first free memory block that is large enough to satisfy the process request.

Working

- Scan memory blocks from the beginning.
- Allocate the first block whose size $\geq$ process size.
- Split the remaining space (if any).

1.Write the c program OS_exp8_1.c

```
#include <stdio.h>
```

```
int main() {
    int blockSize[10], originalBlock[10], processSize[10];
    int blockCount, processCount;
    int allocation[10];
    int totalMemory = 0, totalAllocated = 0, totalFree = 0;

    printf("Enter number of memory blocks: ");
    scanf("%d", &blockCount);

    printf("Enter size of each block:\n");
    for(int i = 0; i < blockCount; i++) {
        scanf("%d", &blockSize[i]);
        originalBlock[i] = blockSize[i]; // Store original size
        totalMemory += blockSize[i];    // Calculate total memory
    }

    printf("Enter number of processes: ");
    scanf("%d", &processCount);

    printf("Enter size of each process:\n");
    for(int i = 0; i < processCount; i++) {
        scanf("%d", &processSize[i]);
        allocation[i] = -1;
    }
}
```

```

// First Fit Allocation
for(int i = 0; i < processCount; i++) {
    for(int j = 0; j < blockCount; j++) {
        if(blockSize[j] >= processSize[i]) {
            allocation[i] = j;
            blockSize[j] -= processSize[i];
            totalAllocated += processSize[i];
            break;
        }
    }
}

// Display Allocation
printf("\nProcess No.\tProcess Size\tBlock No.\n");
for(int i = 0; i < processCount; i++) {
    printf("%d\t%d\t", i+1, processSize[i]);
    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

// Calculate remaining free memory
for(int i = 0; i < blockCount; i++) {
    totalFree += blockSize[i];
}

printf("\nTotal Memory = %d", totalMemory);
printf("\nTotal Allocated Memory = %d", totalAllocated);
printf("\nTotal Free Memory = %d\n", totalFree);

return 0;
}

```

2.Compile and Run

```
gcc OS_exp8_1.c -o OS_exp8_1
./OS_exp8_1
```

3.Expected output

```

Enter number of memory blocks: 5
Enter size of each block:
10 5 030 25 40
Enter number of processes: 4
Enter size of each process:
20 15 30 5

Process No.      Process Size      Block No.
1                20               3
2                15               4
3                30               5
4                5                1

Total Memory = 110
Total Allocated Memory = 70
Total Free Memory = 40

```

2. Best Fit Algorithm

Theory

Definition

The **Best Fit algorithm** allocates the process to the **smallest available memory block** that is large enough to satisfy its requirement.

It tries to reduce wasted memory by choosing the most suitable block.

Working

1. Compare process size with all free memory blocks.
2. Select the block with:
 - Size $\geq$ process size
 - Minimum difference (block size – process size)
3. Allocate memory.
4. Split remaining space (if any).
5. If no suitable block exists $\rightarrow$ process not allocated.

1. Write the c program OS_exp8_2.c

```
#include <stdio.h>
```

```
int main() {
    int blockSize[10], originalBlock[10], processSize[10];
    int blockCount, processCount;
    int allocation[10];
    int totalMemory = 0, totalAllocated = 0, totalFree = 0;

    printf("Enter number of memory blocks: ");
    scanf("%d", &blockCount);

    printf("Enter size of each block:\n");
    for(int i = 0; i < blockCount; i++) {
        scanf("%d", &blockSize[i]);
        originalBlock[i] = blockSize[i]; // store original size
        totalMemory += blockSize[i];    // calculate total memory
    }

    printf("Enter number of processes: ");
    scanf("%d", &processCount);

    printf("Enter size of each process:\n");
    for(int i = 0; i < processCount; i++) {
        scanf("%d", &processSize[i]);
    }
}
```

```

    allocation[i] = -1; // no allocation initially
}

// Best Fit Allocation
for(int i = 0; i < processCount; i++) {
    int bestIndex = -1;

    for(int j = 0; j < blockCount; j++) {
        if(blockSize[j] >= processSize[i]) {
            if(bestIndex == -1 ||
               blockSize[j] < blockSize[bestIndex]) {
                bestIndex = j;
            }
        }
    }

    if(bestIndex != -1) {
        allocation[i] = bestIndex;
        blockSize[bestIndex] -= processSize[i];
        totalAllocated += processSize[i];
    }
}

// Display Allocation
printf("\nProcess No.\tProcess Size\tBlock No.\n");
for(int i = 0; i < processCount; i++) {
    printf("%d\t%d\t", i+1, processSize[i]);
    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

// Calculate Free Memory
for(int i = 0; i < blockCount; i++) {
    totalFree += blockSize[i];
}

printf("\nTotal Memory = %d", totalMemory);
printf("\nTotal Allocated Memory = %d", totalAllocated);
printf("\nTotal Free Memory = %d\n", totalFree);

return 0;
}

```

2.Compile and Run

```
gcc OS_exp8_2.c -o OS_exp8_2
./OS_exp8_2
```

3.Expected output

```
Enter number of memory blocks: 5
Enter size of each block:
10 5 30 25 40
Enter number of processes: 4
Enter size of each process:
20 15 30 5

Process No.      Process Size      Block No.
1                20                4
2                15                3
3                30                5
4                5                 2

Total Memory = 110
Total Allocated Memory = 70
Total Free Memory = 40
```

3.Worst Fit Algorithm

Theory

Definition

The **Worst Fit algorithm** allocates the process to the **largest available memory block** that is large enough to satisfy its requirement.

Instead of minimizing leftover space (like Best Fit), it tries to leave a **large remaining free block** for future processes.

Working

1. Compare process size with all free memory blocks.
2. Select the block with:
 - Size $\geq$ process size
 - Maximum size among all suitable blocks
3. Allocate memory.
4. Split remaining space.
5. If no suitable block exists $\rightarrow$ process not allocated.

1. Write the c program OS_exp8_3.c

```
#include <stdio.h>
```

```
int main() {
    int blockSize[10], originalBlock[10], processSize[10];
    int blockCount, processCount;
    int allocation[10];
    int totalMemory = 0, totalAllocated = 0, totalFree = 0;

    printf("Enter number of memory blocks: ");
    scanf("%d", &blockCount);

    printf("Enter size of each block:\n");
    for(int i = 0; i < blockCount; i++) {
        scanf("%d", &blockSize[i]);
        originalBlock[i] = blockSize[i]; // store original size
        totalMemory += blockSize[i];    // calculate total memory
    }

    printf("Enter number of processes: ");
    scanf("%d", &processCount);

    printf("Enter size of each process:\n");
    for(int i = 0; i < processCount; i++) {
        scanf("%d", &processSize[i]);
        allocation[i] = -1; // initially not allocated
    }

    // Worst Fit Allocation
    for(int i = 0; i < processCount; i++) {
        int worstIndex = -1;

        for(int j = 0; j < blockCount; j++) {
            if(blockSize[j] >= processSize[i]) {
                if(worstIndex == -1 ||
                    blockSize[j] > blockSize[worstIndex]) {
                    worstIndex = j;
                }
            }
        }

        if(worstIndex != -1) {
            allocation[i] = worstIndex;
            blockSize[worstIndex] -= processSize[i];
            totalAllocated += processSize[i];
        }
    }

    // Display Allocation
    printf("\nProcess No.\tProcess Size\tBlock No.\n");
```

```

for(int i = 0; i < processCount; i++) {
    printf("%d\t%d\t", i+1, processSize[i]);
    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

// Calculate total free memory
for(int i = 0; i < blockCount; i++) {
    totalFree += blockSize[i];
}

printf("\nTotal Memory = %d", totalMemory);
printf("\nTotal Allocated Memory = %d", totalAllocated);
printf("\nTotal Free Memory = %d\n", totalFree);

return 0;
}

```

2.Compile and Run

```

gcc OS_exp8_3.c -o OS_exp8_3
./ OS_exp8_3

```

3.Expected output

```

Enter number of memory blocks: 5
Enter size of each block:
10 5 30 25 40
Enter number of processes: 4
Enter size of each process:
20 15 30 5

Process No.      Process Size      Block No.
1                20               5
2                15               3
3                30               Not Allocated
4                5                4

Total Memory = 110
Total Allocated Memory = 40
Total Free Memory = 70

```

Result

The memory management schemes First Fit, Best Fit, and Worst Fit were successfully implemented